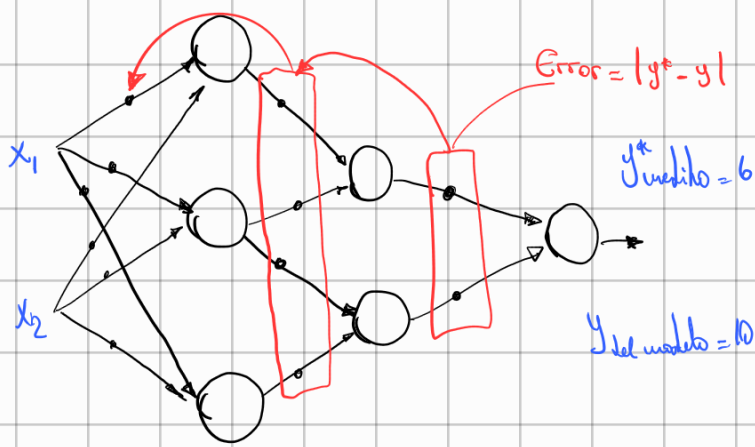




	x_1	x_2	...	x_n
1				
2				
3				
$\vdots$				

Inteligencia es actualizar los pesos de forma que resuelva el credit assignment Problem (CAP)

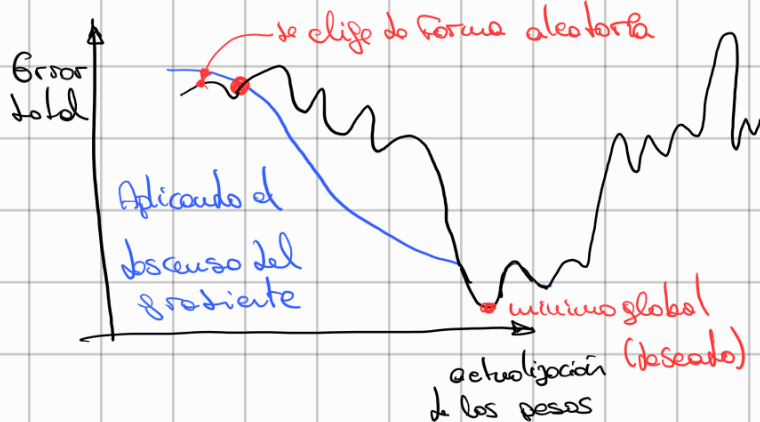
El CAP es el mayor desafío en el aprendizaje cuando una red produce una salida errónea es necesario determinar que pesos de red son responsables. El error debe ser cercano a cero.

¿Porque no corregimos el error para cada medición?

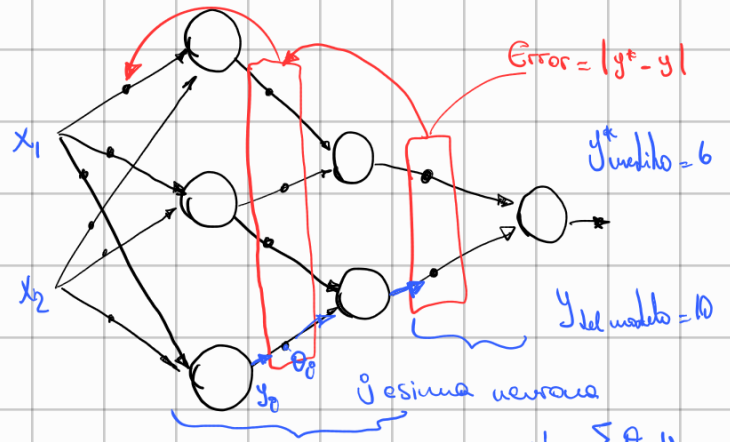
$$x_1^1 \ x_2^1 \ ... \ x_n^1 \rightarrow y^1 \rightarrow \text{error}_1$$

$$x_1^2 \ x_2^2 \ ... \ x_n^2 \rightarrow y^2 \rightarrow \text{error}_2$$

Porque la corrección debe ser sobre el total de los datos o lotes dado que nada nos asegura que al corregir uno de ellos mejorara para los demás y el objetivo es minimizar sobre el error total.



Un método es el descenso del gradiente y en salida controla el gradiente.



$$\theta_{ji}^{(n+1)} = \theta_{ji}^{(n)} + \Delta \theta_{ji}^{(n)} \quad \text{... (1) Aplicando la regla de la cadena a (2) y usando (3)}$$

$$\Delta \theta_{ji} = -n \frac{\partial E}{\partial \theta_{ji}} \quad \text{... (2)}$$

$$n \in [0,1]$$

$$V_j = \sum_{i=1}^n \theta_{ji} y_i \quad \text{... (3)}$$

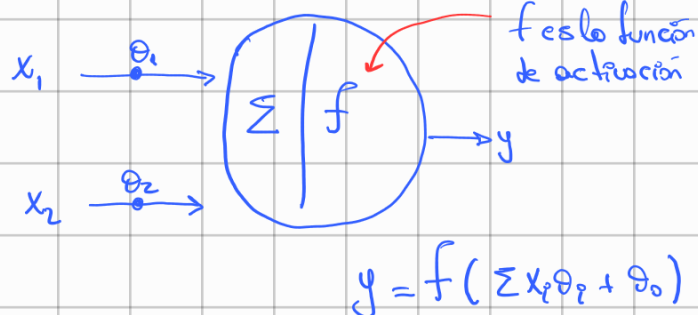
$$\frac{\partial E}{\partial \theta_{ji}} = \frac{\partial E}{\partial V_j} \cdot \frac{\partial V_j}{\partial \theta_{ji}} \quad \text{... (4)}$$

El gradiente de la j-esima neurona

$$\delta_j = -\frac{\partial E}{\partial V_j}$$

el gradiente de $\frac{\partial V_j}{\partial \theta_{ji}} = y_i$

$$\Delta \theta_{ji} = n \delta_j y_i \quad \text{"delta rule"}$$



$$\delta_j = \begin{cases} f'(v_j)(y_j^* - y_j), & \text{Si } j \text{ es una en la capa de salida} \\ f'(v_j) \sum_k \delta_k \theta_{jk}, & \text{Si es una neurona de la capa oculta de la siguiente capa} \end{cases}$$

Para la función de "sigmo" (logística)
 $f'(x) = f(x)(1 - f(x))$

Uniendo todo junto al algoritmo de back propagation quedaría como.

1. $n=1$

Inicializar $\theta(n)$ de forma aleatoria
 Mientras (Criterio de Parada)

Para cada muestra medida (x^*, y^*)

• Correr la red con $\bar{x}$ y obtener

y_{modelo}

• Actualizar los pesos con B.P

fin Para

Fin Mientras

Para aprendizaje
 Supervisado

$$\theta_{je}^{(n+1)} = \theta_{je}^{(n)} + \sum_{X_{\text{training sample}}} \Delta \theta_{je}^X$$

X : lotes de datos tomados por periodos
 (años, meses, días, etc)

Esto facilita encontrar patrones por periodos
 (epoca por epoca)

¿Cómo saber cuando detenerse?

Criterios de Parada

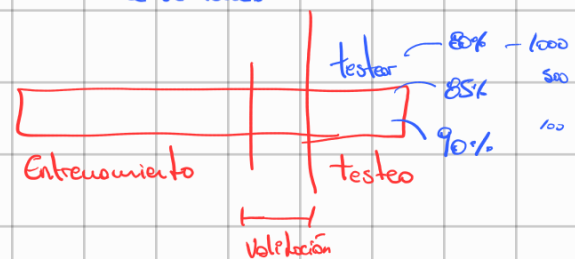
1° Analizar como cambia el error total promedio
 Si el error total no cambia (no hay que perder tiempo de calculo)

La red a convergido ($E \approx 0$) si es cero hay que tener cuidado.

Si el radio absoluto de cambio (Δ) por epoca es suficientemente pequeño.

Generalización based - method

testear para la generalización des pues de cada epoca, si es adecuado se detiene.



Delta rule

$$\Delta \theta_{je} = \eta \delta_j(n) y_{pe}(n)$$

$\eta \rightarrow 0$: no hay aprendizaje

$\eta \rightarrow 1$: hay cambios grandes (es inestable)

$\eta(n)$

$\lambda(n)$

$$\Delta \theta_{je}(n) = \underbrace{\eta \delta_j(n) y_{pe}(n)}_{\text{generalización del Delta rule}} + \underbrace{\lambda \Delta \theta_{je}(n-1)}_{\text{momentum}}$$

Back Propagation (batch mode)

Actualizo los pesos solo despues de que todos las medidas/muestras se han propagado a traves de la red.

topología de la red

Cómo definir el # de capas

" # de neuronas por capa

Esto depende de la tarea a resolver

tamaño del modelo depende en su mayor de prueba y error

1. muy pequeño: underfitting

2. muy grande: overfitting

¿Cómo lo probamos?

